# Labo 06 — Labo pratique : faire survivre des données à leur conteneur

*Objectif : perdre des données une fois pour comprendre, puis les faire survivre avec un volume nommé, un bind mount et un tmpfs ; voir de vos yeux qui possède les fichiers en rootless ; et sauvegarder puis restaurer une base PostgreSQL de deux façons.*

**Prérequis** — Labos 01 à 05 terminés. Images `alpine` et `nginx:alpine` présentes ; `postgres:16-alpine` sera tirée à l'étape 2 (297 Mo).

**Fichiers fournis** (`files/`) — aucun : tout se crée au terminal.

Les sorties ci-dessous sont celles d'une Ubuntu/WSL avec la plage `subuid` standard (`100000:65536`). Les identifiants de conteneurs, les dates et les hash diffèrent chez vous.

---

## Étape 1 — Perdre une donnée

```
mkdir -p ~/labo-docker/06 && cd ~/labo-docker/06
podman run --name scribe alpine sh -c 'echo "note du lundi" > /data.txt; ls -l /data.txt'
podman diff scribe
```

**Observez** `-rw-r--r-- 1 root root 14 … /data.txt`, puis `A /data.txt` et `C /etc` : la couche writable du conteneur `scribe` contient un fichier ajouté et un répertoire modifié.

```
podman run --rm alpine ls /data.txt
podman start -a scribe
podman rm scribe
```

**Observez** `ls: /data.txt: No such file or directory` pour le nouveau conteneur : l'image n'a pas changé. Puis `scribe` redémarré affiche bien son fichier : `stop` / `start` conservent la couche. Puis `rm` : le fichier n'existe plus nulle part.

*Explication.* Chaque conteneur a sa propre couche writable, invisible des autres, détruite avec lui. Une donnée qui doit survivre ne peut pas y vivre.

---

## Étape 2 — Un volume nommé pour PostgreSQL

```
podman volume create pgdata
podman volume inspect pgdata
```

**Observez** `"Mountpoint": "/home/<vous>/.local/share/containers/storage/volumes/pgdata/_data"`, `"NeedsCopyUp": true`, `"MountCount": 0` : un répertoire vide, géré par Podman, encore monté nulle part.

```
podman run -d --name db -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data
postgres:16-alpine
sleep 5 ; podman logs db 2>&1 | tail -1
podman exec db psql -U postgres -c "CREATE TABLE clients(id serial, nom text); INSERT INTO
clients(nom) VALUES ('Durand'),('Peeters');"
podman exec db psql -U postgres -c "SELECT * FROM clients;"
podman inspect db --format '{{json .Mounts}}'
```

**Observez** `database system is ready to accept connections`, `CREATE TABLE` / `INSERT 0 2`, les deux lignes, puis le montage : `"Type":"volume","Name":"pgdata",…` `"Destination":"/var/lib/postgresql/data","RW":true`.

```
podman rm -f -t 0 db
podman volume ls
podman run -d --name db2 -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data
postgres:16-alpine
sleep 5 ; podman exec db2 psql -U postgres -c "SELECT * FROM clients;"
podman rm -f -t 0 db2
```

**Observez** `pgdata` toujours listé après la suppression du conteneur, et le second conteneur qui retrouve `Durand` et `Peeters`.

*Explication.* Le conteneur est parti, le volume est resté. Le second conteneur a trouvé un `PGDATA` déjà initialisé et n'a rien recréé — `POSTGRES_PASSWORD` n'a d'effet qu'au tout premier démarrage.

---

## Étape 3 — Le volume anonyme qu'on n'a pas demandé

```
podman image inspect postgres:16-alpine --format '{{.Config.Volumes}}'
podman run -d --name anon -e POSTGRES_PASSWORD=secret postgres:16-alpine
sleep 3 ; podman volume ls
podman inspect anon --format '{{range .Mounts}}{{.Type}} {{.Name}} -> {{.Destination}}{{end}}'
podman rm -f -t 0 anon
podman volume ls
```

**Observez** `map[/var/lib/postgresql/data:{}]` : l'image déclare un `VOLUME`. Sans `-v`, Podman a créé un volume au nom de 64 caractères hexadécimaux, monté au même endroit. Après `rm`, il est **toujours là**.

```
podman volume ls --filter dangling=true
```

**Observez** que le filtre liste le volume anonyme… **et** `pgdata`. « Dangling » signifie « monté par aucun conteneur existant » — et depuis la suppression de `db2`, c'est le cas de `pgdata`, qui contient pourtant vos clients. Passer cette liste à `volume rm` (ou lancer `podman volume prune`) les détruirait tous les deux.

```
podman volume rm $(podman volume ls --filter dangling=true -q | grep -v '^pgdata$')
podman run -d --name anon2 -e POSTGRES_PASSWORD=secret postgres:16-alpine
sleep 3 ; podman rm -f -t 0 -v anon2
podman volume ls
```

**Observez** qu'il ne reste que `pgdata` : le premier volume anonyme a été supprimé nommément, le second avec son conteneur grâce à `rm -v`.

*Explication.* `podman rm` ne supprime pas les volumes ; `podman rm -v` supprime les volumes **anonymes** du conteneur (jamais les nommés). Le filtre `dangling=true` ne distingue pas un déchet d'un volume précieux entre deux conteneurs : lisez la liste, puis supprimez par nom.

---

## Étape 4 — Qui possède les fichiers ?

```
ls -ln ~/.local/share/containers/storage/volumes/
ls ~/.local/share/containers/storage/volumes/pgdata/_data
```

**Observez** `drwx------ 3 1000 1000 … pgdata` : le répertoire du volume est à vous. Mais son contenu : `ls: cannot open directory … Permission denied`.

```
podman unshare cat /proc/self/uid_map
podman unshare ls -ln ~/.local/share/containers/storage/volumes/pgdata/_data | head -4
```

**Observez** la table `0 1000 1` puis `1 100000 65536`, et les fichiers `-rw------- 1 70 70 … PG_VERSION` : dans le namespace, ils appartiennent à l'UID 70, `postgres`.

*Explication.* PostgreSQL écrit sous l'UID 70. Hors du namespace, cet UID est traduit en `100000 + 70 - 1 = 100069` : un utilisateur qui n'est pas vous. `podman unshare` exécute une commande *dans* le namespace, avec la même vue que le conteneur. (Dans le bac à sable de rédaction, la plage subuid commence à 1001 ; le principe est identique.)

---

## Étape 5 — Un bind mount pour nginx

```
mkdir -p html && echo '<h1>Version 1</h1>' > html/index.html
podman run -d --name web -p 18080:80 -v ./html:/usr/share/nginx/html:ro nginx:alpine
sleep 1 ; curl -s localhost:18080
echo '<h1>Version 2</h1>' > html/index.html
curl -s localhost:18080
```

**Observez** `<h1>Version 1</h1>` puis `<h1>Version 2</h1>` : le fichier modifié sur l'hôte est servi immédiatement, sans redémarrage ni rebuild.

```
podman exec web sh -c 'echo "<h1>pirate</h1>" > /usr/share/nginx/html/index.html'
podman inspect web --format '{{json .Mounts}}'
```

**Observez** `sh: can't create …/index.html: Read-only file system`, et dans l'inspect `"Type":"bind","Source":"/home/<vous>/labo-docker/06/html",… "RW":false`.

```
podman run -d --name web-rw -p 18081:80 -v ./html:/usr/share/nginx/html nginx:alpine
podman exec web-rw sh -c 'echo "<h1>Version 3 (ecrite depuis le conteneur)</h1>" >
/usr/share/nginx/html/index.html'
cat html/index.html ; ls -ln html/
podman cp web:/etc/nginx/nginx.conf ./nginx.conf && head -3 nginx.conf
podman rm -f -t 0 web web-rw
```

**Observez** `Version 3` dans **votre** fichier, qui appartient toujours à `1000 1000` : écrit par le root du conteneur, c'est-à-dire vous. Et `podman cp` a extrait la configuration nginx sans montage.

*Explication.* Un bind mount, c'est le même répertoire vu de deux endroits. `:ro` protège vos fichiers ; sans lui, le conteneur écrit chez vous avec l'UID correspondant à son utilisateur.

---

## Étape 6 — Le piège de l'UID

```
mkdir -p sortie
podman run --rm -v ./sortie:/out alpine sh -c 'id -u; echo bonjour > /out/root.txt'
ls -ln sortie
podman run --rm --user 1000 -v ./sortie:/out alpine sh -c 'id -u; echo bonjour > /out/user.txt'
```

**Observez** `0` et un fichier `root.txt` qui appartient à `1000 1000`. Puis `1000` et `sh: can't create /out/user.txt: Permission denied`.

```
podman run --rm --user 1000 -v ./sortie:/out:U alpine sh -c 'id -u; echo bonjour >
/out/user.txt; ls -ln /out'
ls -ln sortie
```

**Observez** dans le conteneur `-rw-r--r-- 1 1000 0 … user.txt`, et sur l'hôte `-rw-r--r-- 1 100999 1000 … root.txt` et `user.txt` : `:U` a changé le propriétaire **de tout le dossier** vers l'UID du conteneur, qui n'est plus vous.

```
cat sortie/user.txt
podman unshare chown -R 0:0 sortie && ls -ln sortie
```

**Observez** que la lecture fonctionne (droits `r` pour tous), puis `1000 1000` : le dossier est de nouveau à vous.

*Explication.* Le root du conteneur, c'est vous ; l'UID 1000 du conteneur, c'est 100999 sur l'hôte. `:U` fait un `chown` réel et persistant ; `podman unshare chown 0:0` le défait, puisque root dans le namespace = 1000 dehors.

---

## Étape 7 — tmpfs et syntaxe `--mount`

```
podman run --rm --tmpfs /scratch:size=64m alpine sh -c 'df -h /scratch; echo temporaire >
/scratch/f.txt; ls /scratch'
podman run --rm --mount type=tmpfs,destination=/scratch,tmpfs-size=64m alpine df -h /scratch
```

**Observez** `tmpfs 64.0M 0 64.0M 0% /scratch` et `f.txt` : un disque en mémoire de 64 Mo, qui n'existera plus à la ligne suivante.

```
podman run --rm --mount type=volume,source=pgdata,destination=/data alpine ls /data | head -3
podman run --rm --mount type=bind,source=./html,destination=/html,ro alpine sh -c 'cat
/html/index.html; touch /html/x'
podman run --rm -v ./html:/html:ro,Z alpine cat /html/index.html
```

**Observez** `PG_VERSION`, `base`, `global` (le volume vu par un autre conteneur), `Read-only file system` au `touch`, et `:Z` accepté sans rien changer.

*Explication.* `--mount` dit explicitement ce qu'on monte ; `-v` le devine à la forme de la source. `:Z` ne fait rien sans SELinux, mais prépare vos commandes aux serveurs RHEL.

---

## Étape 8 — Sauvegarder, casser, restaurer

```
podman run -d --name db -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data
postgres:16-alpine
sleep 5 ; podman exec db pg_dump -U postgres > dump.sql
ls -lh dump.sql ; grep -A3 "COPY public.clients" dump.sql
podman volume rm pgdata
```

**Observez** un `dump.sql` d'environ 1,8 Ko contenant `COPY public.clients (id, nom) FROM stdin;` suivi des deux lignes : une sauvegarde logique, faite à chaud. Puis `Error: volume pgdata is being used by the following container(s): … volume is being used`.

```
podman stop -t 5 db
podman volume export pgdata -o pgdata.tar
ls -lh pgdata.tar ; tar tf pgdata.tar | head -3
podman volume create pgdata-copie
podman volume import pgdata-copie pgdata.tar
podman run -d --name db-copie -e POSTGRES_PASSWORD=secret -v pgdata-
copie:/var/lib/postgresql/data postgres:16-alpine
sleep 5 ; podman exec db-copie psql -U postgres -c "SELECT * FROM clients;"
```

**Observez** une archive de 39 Mo (`PG_VERSION`, `base/`, …) et, dans le volume importé, les deux clients : sauvegarde physique, conteneur arrêté.

```
podman volume create pgdata-vierge
podman run -d --name db-vierge -e POSTGRES_PASSWORD=secret -v pgdata-
vierge:/var/lib/postgresql/data postgres:16-alpine
sleep 5 ; podman exec db-vierge psql -U postgres -c "SELECT * FROM clients;"
podman exec -i db-vierge psql -U postgres -q < dump.sql
podman exec db-vierge psql -U postgres -c "SELECT * FROM clients;"
```

**Observez** `ERROR: relation "clients" does not exist` sur la base vierge ; le rejeu du dump n'affiche que deux petits résultats (`set_config`, `setval`) ; puis les deux clients sont de retour.

```
podman run --rm -v pgdata:/data:ro -v ./:/backup alpine tar czf /backup/pgdata.tgz -C /data .
ls -lh pgdata.tgz
podman system df -v | sed -n '/Local Volumes/,$p'
```

**Observez** `pgdata.tgz` (4,5 Mo compressés) écrit dans votre dossier par un conteneur jetable, et les trois volumes d'environ 40 Mo chacun.

*Explication.* `pg_dump` lit un instantané cohérent pendant que la base tourne ; la copie de fichiers exige l'arrêt. `volume export` / `import` sont propres à Podman ; le conteneur Alpine avec deux montages est la méthode qui marche partout — et la seule qui lise les fichiers de l'UID 100069 sans `unshare`.

---

## Nettoyage

```
podman rm -f -t 0 db db-copie db-vierge
podman volume rm pgdata pgdata-copie pgdata-vierge
podman volume ls
podman rmi docker.io/library/postgres:16-alpine
rm -rf ~/labo-docker/06/sortie ~/labo-docker/06/pgdata.tar ~/labo-docker/06/pgdata.tgz
```

**Observez** `podman volume ls` vide. Gardez `html/` et `dump.sql` : inutiles aux labos suivants, mais ils tiennent 2 Ko. `postgres:16-alpine` sera retirée au labo 07 — sa suppression ici évite de garder 297 Mo pour rien, et le labo 07 la retire de nouveau.

---

## Ce que vous devez pouvoir affirmer maintenant

- `stop` / `start` conservent la couche writable ; `rm` la détruit : `No such file or directory`.
- Un volume nommé survit au conteneur : `db2` retrouve les clients de `db`.
- `VOLUME` dans l'image = volume anonyme de 64 caractères si on oublie `-v` ; `rm -v` l'emporte, et `dangling=true` liste aussi vos volumes nommés au repos.
- Les fichiers de PostgreSQL appartiennent à `70` dans le namespace, `100069` dehors : `Permission denied` sans `podman unshare`.
- Un bind mount `:ro` est servi en direct et refuse l'écriture : `Read-only file system`.
- `--user 1000` ne peut pas écrire dans votre dossier ; `:U` le lui donne, `podman unshare chown` le reprend.
- `pg_dump` à chaud, `volume export` ou `tar` conteneur arrêté ; `volume rm` refuse tant qu'un conteneur monte le volume.